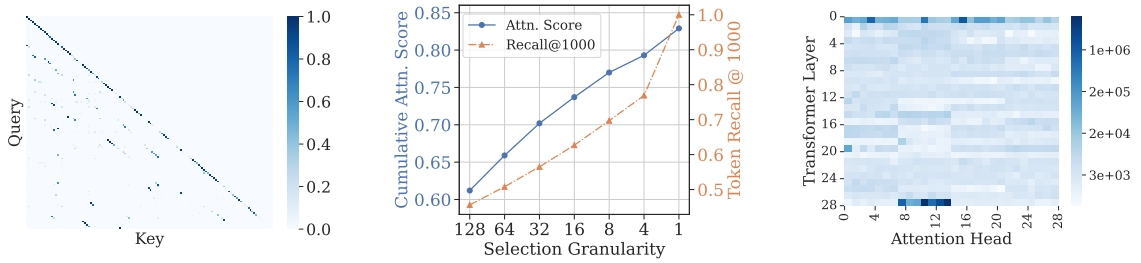
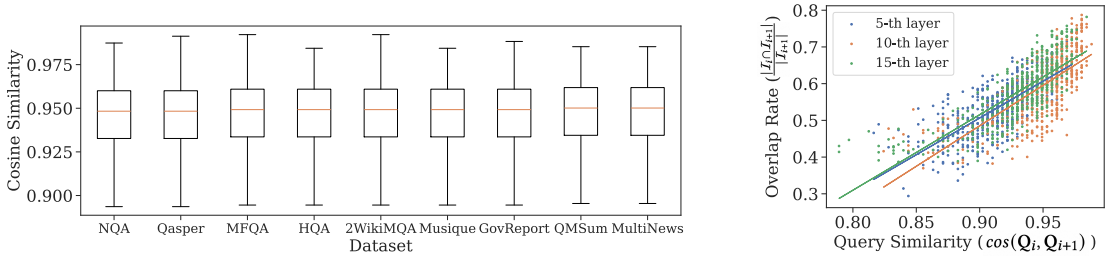


(a) Attention is sparse in token-level. (b) Block-level selection is sub-optimal. (c) Attention logits is head-distinctive.

Figure 2: Motivations for token-level selection. (a) Visualization of attention scores sparsity. (b) Attention scores and critical token recalled by 1K token budget. (c) The L_1 norm of attention logits in each attention head.



(a) Consecutive queries show consistent similarity patterns across datasets. (b) Selection overlaps with similar queries.

Figure 3: Observations on similarity of consecutive queries. (a) Cosine similarity distribution between consecutive queries. (b) The token selection overlap rate ($\frac{|\mathcal{I}_i \cap \mathcal{I}_{i+1}|}{|\mathcal{I}_{i+1}|}$) with respect to consecutive Query similarity.

4 Motivations and Observations

Attention is Sparse, Non-contiguous and Head-Distinctive. Previous works on long-context inference have demonstrated the sparsity of attention scores in LLMs, particularly when processing long texts. Recent approaches (*e.g.*, InfLLM, QUEST and MInference) partition the KV Cache into non-overlapping blocks, estimating block criticality for sparse attention calculations. These methods assume that critical tokens tend to be contiguous. However, our further observations reveal that this assumption does not always hold true in practice. As illustrated in *Fig.* 2a, attention scores are sparsely distributed at the token-level. This non-contiguity leads to significant omissions in block-level token selection. *Fig.* 2b demonstrates that finer selection granularity improves recall of critical tokens, motivating us to perform token-level selection. For token-level selection, an intuitive approach would be to directly select the top-k tokens with largest attention logits. However, *Fig.* 2c reveals considerable disparity in the L_1 norm of attention logits across attention heads. As a result, the selection result tends to be dominated by a few heads with disproportionately large attention logits, driving us to design a more robust selection function that maintains the independence of heads.

Consecutive Queries are Similar. As sparsity of attention is dynamic (Jiang et al., 2024), token selection should be performed for every Query, which inevitably increases the computational overhead of selective sparse attention. Fortunately, we observe that consecutive Queries exhibit high similarity, as shown in *Fig.* 3a. Intuitively, when two consecutive Queries are highly similar, their dot products with the Keys will also be similar, leading to substantial overlap in the token selection results. Due to space constraints, we provide an informal lemma about this below. The formal version and corresponding proof can be found in the Appendix C.

Lemma 1 *(Informal). Consider Queries* $\mathbf{Q}_1, \mathbf{Q}_2 \in \mathbb{R}^{1\times d}$ *that are consecutive and a Key set* $\{\mathbf{K}_i\}_{i=1}^{N}$*. Let* $\mathcal{I}_1$*, and* $\mathcal{I}_2$ *be the sets of indices of the top-*k *Keys selected by dot product for* $\mathbf{Q}_1$*, and* $\mathbf{Q}_2$ *respectively. If* $cos(\mathbf{Q}_1, \mathbf{Q}_2) > \epsilon$*, where* ϵ *is a threshold, then* $\mathcal{I}_1 = \mathcal{I}_2$*.*

Fig. 3b illustrates this lemma experimentally. It shows that the overlap rate of token selection tends to increase with Query similarity. This key insight motivates us to reuse selection results for similar queries, improving computational efficiency. Moreover, the similarity distribution of consecutive

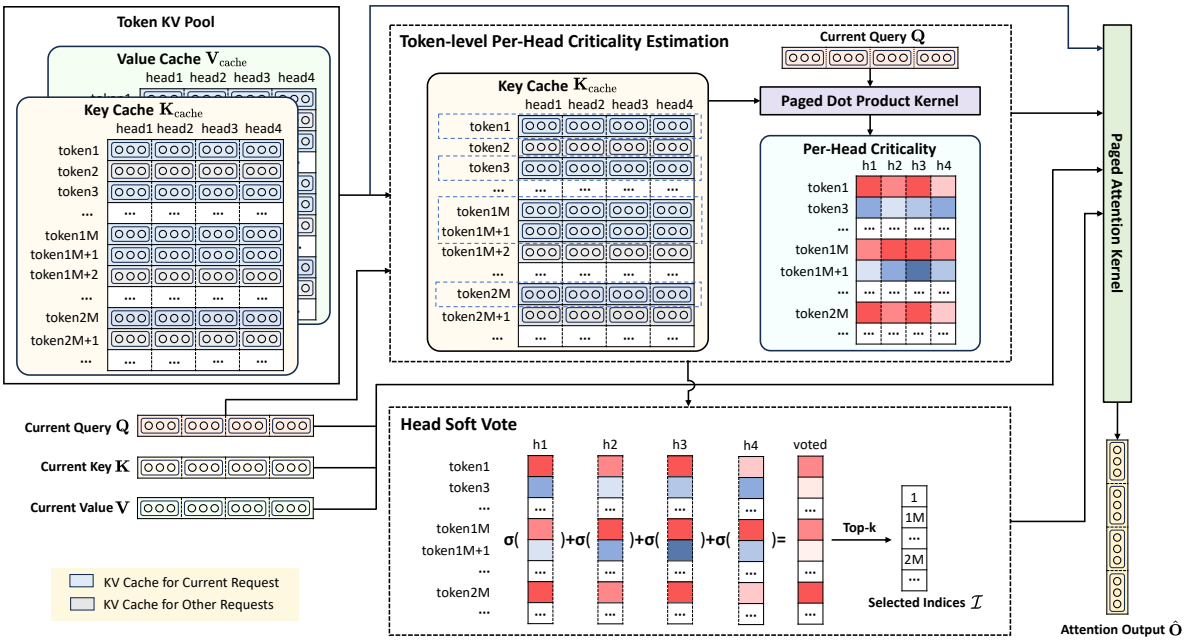


Figure 4: Execution flow of *TokenSelect*: 1) calculate per-head criticality via Paged Dot Product Kernel; 2) perform head soft vote to obtain selected indices; 3) execute selective sparse attention via Paged Attention Kernel.

Queries remains consistent across different tasks, as demonstrated in *Fig.* 3a, allowing us to apply a global similarity threshold across all scenarios.

5 Designs of *TokenSelect*

In this section, we will introduce the design details of *TokenSelect*, primarily encompassing the Selection Function, the Selection Cache, and efficient implementation of *TokenSelect*. The overall workflow of *TokenSelect* is illustrated in *Fig.* 4.

5.1 Selection Function

The simplest selection function is to determine the criticality of the tokens via the dot product of $\mathbf{Q}$ and $\mathbf{K}_{\text{cache}}$, then select the top-$k$ as $\mathbf{K}_{\text{select}}, \mathbf{V}_{\text{select}}$. The selected indices $\mathcal{I}$ are calculated as:

$$\mathcal{I}_{\text{topk}} = \text{TopK}\left(\mathbf{Q} \cdot \mathbf{K}_{\text{cache}}^{h\top}\right). \tag{5}$$

However, as discussed in *Sec.* 4, this approach is prone to inaccuracies due to disparities in norm of attention logits between heads. To maintain independence between heads, a better approach is to have each head select the top-k most critical tokens, and then determine the final selection through voting among the heads, where $\mathbb{I}$ is indicator function:

$$\mathcal{I}_{\text{head-vote}} = \text{TopK}\left(\sum_{h=1}^{H} \mathbb{I}\left(i \in \text{TopK}\left(\mathbf{Q}^h \cdot \mathbf{K}_{\text{cache}}^{h\top}\right)\right)\right) \tag{6}$$

Unfortunately, despite better performance, this method relies on `scatter_add` and multiple `topk` operations, resulting in low efficiency on GPUs. Additionally, the 0/1 voting ignores the relative importance of tokens for each head. Therefore, we propose a head soft vote approach that offers better performance and efficiency. Specifically, we first calculate the per-head criticality, then normalize through softmax, and sum the results for all heads:

$$\mathcal{I}_{\text{head-soft-vote}} = \text{TopK}\left(\sum_{h=1}^{H} \sigma\left(\mathbf{Q}^h \cdot \mathbf{K}_{\text{cache}}^{h\top}\right)\right). \tag{7}$$

5.2 Optimizing Selection Frequency

Although the aforementioned selection function can reduce the complexity of attention from $O(N^2)$ to $O(k^2), k \ll N$, while maintaining performance, the execution time of the selection function itself still affects the latency of inference. To further accelerate long-context inference, based on our observations of the similarity of consecutive Queries, we design optimization strategies for both the prefill stage and the decode stage to reduce the selection frequency while ensuring its effectiveness.

In the prefill stage, $\mathbf{Q}_{\text{prefill}} \in \mathbb{R}^{n_{\text{in}} \times d}$ is inputed. In long-context scenarios, the number of tokens in the user's input sequence n_{in} may reach up to 1M, making it impractical to perform selection for each Query token. Considering the similarity of consecutive Queries, we use chunk-wise token selection, inputting $\frac{1}{c}\sum_{i=1}^{c}(\mathbf{Q}_C)_i$ into the selection function, where $\mathbf{Q}_C \in \mathbb{R}^{c \times d}$ is the Query chunk and c is the chunk size. This method helps maintain the compute-intensive nature of the prefill stage, preventing it from becoming memory bound.

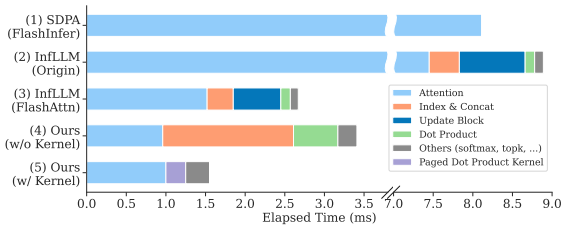


Figure 5: Time breakdown for single chunk prefill step under different attention implementations (chunk size: 512, KV Cache length: 128K, attended tokens: 4K).

In the decode stage, due to the auto-regressive characteristic of LLMs, we need to frequently perform selection for $\mathbf{Q}_{\text{decode}}$, and this process cannot be executed chunk-wise like in the prefill stage. To reduce the frequency of token selection in the decode stage, we propose the Selection Cache. Consecutive similar Queries will hit the cache, thereby directly loading the cached selection results for the previous Query. The Selection Cache allows us to reduce decode latency while maintaining the performance. The formal formulation of the Selection Cache is detailed in Algorithm 1.

5.3 Efficient Implementation

To ready *TokenSelect* for real-world use, efficient implementation is crucial. We first analyze the time breakdown of representative block-level selective sparse attention method, InfLLM (Xiao et al., 2024a). From (1)(2)(3) in *Fig.* 5, we can observe that, despite lowering theoretical complexity, actual runtime depends heavily on implementation. The incompatibility with efficient attention implementations such as Flash Attention has resulted in methods requiring historical attention scores (*e.g.*, H_2O, TOVA, SnapKV, InfLLM) impractical in real-world serving. Analysis of InfLLM's Flash Attention–compatible version shows that, although block-level criticality estimation aims to cut selection overhead, the dot product isn't the main bottleneck. Instead, indexing and coalescing selected KV Cache tokens in GPU memory (HBM)—during block updates and KV Cache concatenation—incurs heavy I/O, aggravating LLM inference's memory-bound limits. Based on this, we propose that Paged Attention is a more suitable implementation for selective sparse attention. Using paged KV Cache management (with page size=1 for *TokenSelect*), we can reduce the I/O volume for selection results from the scale of all selected KV Caches $O(2kd)$ to the scale of their indices $O(k)$. However, (4) in *Fig.* 5 reveals another bottleneck under paged KV Cache management. Since logically contiguous KV Cache is not entirely contiguous in HBM, it also needs to be made contiguous before performing selection operations. To address this issue, we design a Paged Dot Product Kernel using Triton, which significantly improves the overall efficiency of *TokenSelect*. The formal description of this kernel is detailed in Algorithm 2.

6 Experiments

In this section, we introduce the experimental setup and evaluate the performance and efficiency of our *TokenSelect* on long-context inference benchmarks.

6.1 Experimental Settings

Datasets. To evaluate *TokenSelect*'s performance on long-context inference, we use three representative datasets: InfiniteBench (Zhang et al., 2024a), RULER (Hsieh et al., 2024), and LongBench (Bai et al., 2024). Detailed descriptions and the evaluation metrics used are provided in Appendix G.

Baselines. To conduct a comprehensive evaluation of *TokenSelect*'s performance, we carry out benchmarks on three mainstream open-source LLMs-`Qwen2-7B-Instruct` (Yang et al., 2024a), `Llama-3-8B-Instruct` (Dubey et al., 2024), and `Yi-1.5-6B-Chat` (AI et al., 2024)-comparing against the following state-of-the-art long-context inference methods: NTK-scaled RoPE, SelfExtend, StreamingLLM, InfLLM, *SnapKV*, *InfiniGen*, *QUEST*, *RetrievalAttention* and *MInference*. Detailed descriptions of these methods are provided in Appendix F. It is worth noting that the methods indicated in *italics* lack length-extrapolation capability; thus, we evaluate them using an alternative approach, applying them to `Llama-3-8B-Instruct-262k` (long-text post-trained `Llama-3-8B-Instruct`).

Implementation details. In all experiments in this paper, we employ greedy decoding to ensure the reliability of the results. For our *TokenSelect*, we implement it on SGLang (Zheng et al., 2024), which is a fast serving framework based on Flasherinfer (flashinfer ai). We implement our method using PyTorch (Paszke et al., 2019) and Triton (Tillet et al., 2019). We follow the baseline approach, including 128 initial tokens and n_{local} most recent tokens in the attention computation in addition to the k selected tokens. For NTK and SelfExtend, we extend the model's context length to 128K. For